Tugas Besar M1
IF4031 Arsitektur Aplikasi Terdistribusi



Ditulis Oleh:

AkuAdalahBMKG

13523035	M. Rayhan Farrukh
13523073	Alfian Hanif Fitria Yustanto
13523091	Carlo Angkisan

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

Daftar Isi

Daftar Isi 1
1. Bagian Umum 2
    1.1. Tabel Kontribusi 2
    1.2. Deskripsi Umum Sistem 2
    1.3. Diagram Arsitektur Sistem 4
    1.4. Daftar Teknologi Terpilih 5
    1.5. Asumsi 5
    1.6. Petunjuk Menjalankan Sistem 6
2. Jabaran Problem Set 6
    2.1. Problem 1: Arsitektur & Interoperabilitas Data 6
    2.2. Problem 2: Concurrency, Availability, & Graceful Degradation 11
    2.3. Problem 3: Service-to-Service Authentication & Trust 18
    2.4. Problem 4: Arsitektur Microservice & Flexible Storage 26
    2.5. Problem 5: Fan-out Event Distribution/Pub-sub 33
3. Lampiran 39
    3.1. Penggunaan LLM 39

1. Bagian Umum

1.1. Tabel Kontribusi

NIM	Nama	Kontribusi
13523035	M. Rayhan Farrukh	**Perancangan & Implementasi** ● Setup awal repositori ● Mock BMKG dan PVMBG ● Utilitas shared ● Consumer notifier ● Load testing **Laporan** ● 1.4 Daftar Teknologi Terpilih ● 1.5 Asumsi ● 1.6 Petunjuk Menjalankan Sistem ● 2.1 Problem 1
13523073	Alfian Hanif Fitria Yustanto	**Perancangan & Implementasi** ● Aggregator ● Canonical store ● Publisher ke message broker **Laporan** ● 1.3 Diagram Arsitektur Sistem ● 2.2 Problem 2 ● 2.4 Problem 4
13523091	Carlo Angkisan	**Perancangan & Implementasi** ● Auth service dan auth store ● Client-facing API ● Message broker ● Consumer dashboard **Laporan** ● 1.2 Deskripsi Umum Sistem ● 2.3 Problem 3 ● 2.5 Problem 5

1.2. Deskripsi Umum Sistem

Informasi kebencanaan di Indonesia bersumber dari beberapa instansi yang bekerja secara otonom. BMKG memantau gempa bumi dan potensi tsunami. PVMBG memantau aktivitas gunung api. Setiap instansi memiliki format data, mekanisme autentikasi, dan karakteristik layanan sendiri. Layanan BMKG cenderung cepat dan

stabil, sedangkan PVMBG lebih lambat, fluktuatif, dan sesekali tidak tersedia. Di sisi lain, BNPB sebagai lembaga koordinasi membutuhkan gambaran bahaya yang utuh dari kedua sumber tersebut untuk diteruskan ke berbagai pihak.

Sistem yang kami bangun adalah platform koordinasi kebencanaan dengan BNPB sebagai simpul koordinasi. Platform ini mengambil data dari BMKG dan PVMBG, menyatukannya ke dalam satu format kanonik bernama *HazardEvent*, lalu menyalurkannya ke *client downstream*. Alur utamanya adalah sebagai berikut:

1. **Ingest**
   Aggregator melakukan *polling* berkala ke BMKG dan PVMBG dengan interval yang dapat dikonfigurasi. Data dari kedua sumber dipetakan ke *HazardEvent* sesuai aturan pemetaan, lalu disimpan di *Canonical Store*.
2. **Distribusi**
   Setiap *HazardEvent* baru dipublikasikan ke *message broker*. *Consumer* seperti dashboard dan notifier berlangganan event tersebut tanpa perlu dikenal oleh aggregator.
3. **Akses client**
   Client downstream, yaitu Media/Pers, Tim Lapangan, dan BNPB Pusat/Internal Ops, membaca data melalui *Client-Facing API*. Access token diperoleh dari *Auth Service*. Hak akses dibedakan berdasarkan scope, dimana Media hanya menerima field berklasifikasi Ringkasan, sedangkan Tim Lapangan dan Ops menerima seluruh field termasuk data Mentah.

Sistem disusun sebagai kumpulan microservice. Setiap service berjalan di container Docker tersendiri dan berkomunikasi hanya melalui jaringan (REST dan message broker). Setiap penyimpanan dimiliki tepat satu service. Jalur baca client dipisahkan dari jalur ingest, sehingga client tidak pernah menunggu respons PVMBG secara langsung. Dengan demikian, keterlambatan atau matinya satu sumber tidak membuat seluruh layanan ikut terganggu.

Pada Milestone 1, sistem ini mencakup dua instansi sumber (BMKG dan PVMBG) dan dua jenis bahaya (seismik dan vulkanik). Kedua instansi disimulasikan dengan *mock service* sesuai kontrak spesifikasi, termasuk simulasi *outage* dan perubahan skema pada PVMBG. Di sisi BNPB, sistem terdiri atas Aggregator, Canonical Store, Client-Facing API, Auth Service, Message Broker, serta consumer dashboard dan notifier. Sistem melayani tiga client dengan hak akses berbeda dan dirancang untuk menyelesaikan problem P1–P5. Seluruh client bersifat *read-only*, sehingga jalur pelaporan dari lapangan ke BNPB berada di luar cakupan.

1.3. Diagram Arsitektur Sistem

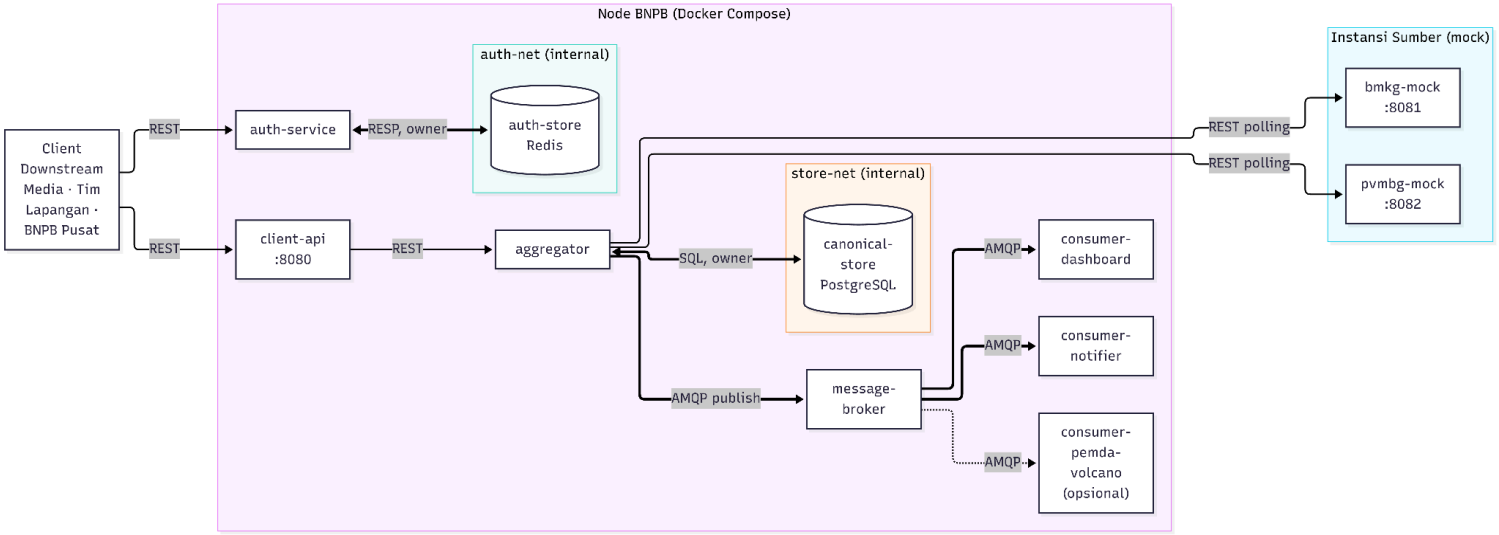


Diagram di atas menunjukkan seluruh container dalam satu Docker Compose. Setiap kotak adalah satu container, dan Client Downstream berada di luar batas node BNPB karena hanya berperan sebagai pemanggil, bukan service.

- **Protokol komunikasi** Seluruh jalur permintaan memakai REST: client ke Auth Service dan Client-Facing API, Client-Facing API ke API internal Aggregator, serta polling Aggregator ke BMKG dan PVMBG. Distribusi event memakai AMQP melalui message broker (RabbitMQ). Aggregator hanya mempublikasikan ke broker, sedangkan consumer menerima event dari antrean masing-masing.
- **Kepemilikan penyimpanan dan batas jaringan.** Canonical Store (PostgreSQL) hanya dapat diakses oleh Aggregator, dan auth-store (Redis, penyimpan refresh token) hanya oleh Auth Service. Masing-masing berada di network Docker terisolasi (`store-net` dan `auth-net`), sehingga tidak dapat dijangkau dari host maupun service lain. Client-Facing API membaca data hanya lewat API Aggregator.
- **Consumer.** Consumer-dashboard dan consumer-notifier subscribe pada stream yang sama secara independen. Consumer-pemda-volcano bersifat opsional dan hanya menerima event vulkanik. Customer ini ditambahkan untuk mendemonstrasikan penambahan consumer baru tanpa mengubah kode Aggregator.
- **Ketahanan.** Karena polling terpisah dari pelayanan client, saat PVMBG lambat atau mati client tetap menerima data terakhir dari Canonical Store beserta penanda data stale (`stale_since`). Broker menahan event untuk consumer yang sedang mati dan mengirimkannya setelah consumer pulih, tanpa menghambat Aggregator.

1.4. Daftar Teknologi Terpilih

Kategori	Teknologi
Bahasa Pemrograman	Go
Canonical Store (DB)	PostgreSQL 16
Message Broker	RabbitMQ 4
Auth	JWT
Token Store	Redis 7
Load Testing	Custom runner & k6

Alasan Pemilihan Teknologi:

- **Go**: Goroutine memudahkan pemrosesan secara paralel serta menahan beban konkuren. Serta Hasil build berupa satu binary kecil sehingga tiap container mudah di-rebuild sendiri.
- **PostgreSQL:** Kolom `attributes` bertipe JSONB menampung atribut dinamis seperti `confidence_level` tanpa migrasi skema, dan record lama serta baru tetap terbaca berdampingan. Field kanonik tetap berbentuk kolom relasional.
- **RabbitMQ:** Queue durable dengan manual ack menahan event saat consumer mati dan mengirimkannya setelah pulih tanpa menghambat producer. Consumer baru cukup di-bind ke exchange tanpa mengubah kode Aggregator.
- **JWT:** Token membawa scope dan TTL pendek (default 60 detik) yang ditegakkan di sisi server. Tanda tangan asimetrik memungkinkan Client-Facing API memverifikasi token secara lokal tanpa memanggil Auth Service.
- **Redis:** Menyimpan hash refresh token untuk rotasi dan deteksi pemakaian ulang, sehingga token lama ditolak setelah refresh dengan builtin features pada Redis.
- **K6 & custom runne**r: k6 dapat mengatur ≥50 koneksi paralel selama ≥60 detik dan melaporkan throughput serta p50/p95/p99 dan digunakan karena kemudahan penggunaannya. Runner digunakan untk membungkus skenario agar mudah diulang saat demo.

1.5. Asumsi

1. Kehilangan data selama sumber tidak aktif dapat ditoleransi. Polling inkremental dengan *`since`* tidak mengambil ulang rentang yang terlewat.

2. Client yang bertanggung jawab atas retry. Saat kapasitas penuh, server cukup menjawab 429 dengan `Retry-After` tetap 1 detik, tanpa antrean maupun backoff adaptif.
3. Endpoint admin pada Mock PVMBG menggunakan kredensial sama dengan endpoint GET.
4. Sumber hanya menambah field pada skema dan tidak pernah mengubah tipe atau menghapus field yang sudah ada (additive only).
5. Isolasi jaringan Docker (`internal: true`) dianggap cukup sebagai pengaman kanal internal. Panggilan Client-Facing API ke Aggregator dan koneksi Auth Service ke Redis tidak perlu memakai kredensial tambahan karena ini.

1.6. Petunjuk Menjalankan Sistem

Seluruh sistem dijalankan dengan Docker Compose dari kondisi awal. Berikut tahapan menjalankan sistem:

- Clone repositori
- Copy seluruh .env.example menjadi .env, pastikan variabel yang digunakan antar service konsisten
- Generate keypair Ed25519 untuk JWT
- docker compose up --build -d.

Langkah lengkap beserta perintahnya dapat dilihat pada README.md.

Setelah semua container berjalan, status tiap service dapat diperiksa melalui endpoint /health masing-masing.

Port default:

- Client-Facing API 8080
- BMKG 8081
- PVMBG 8082
- Auth Service 8084
- Aggregator (8083) dan kedua consumer (8085, 8086) hanya dapat diakses dari dalam jaringan Docker
- RabbitMQ Management tersedia di port 15672

2. Jabaran Problem Set

2.1. Problem 1: Arsitektur & Interoperabilitas Data

2.1.1. Rancangan Solusi

Permasalahan utama pada Problem 1 adalah keterikatan gateway pada skema data yang tetap. PVMBG akan menambahkan field baru tanpa koordinasi, sehingga pemetaan yang bersifat hardcoded berisiko kehilangan

data baru atau gagal mem-parsing payload. Untuk mengatasinya, jalur ingestiion Aggregator dirancang agar tahan terhadap perubahan skema. Field baru diparsing tanpa perubahan kode, restart, dan crash, serta memiliki kebijakan handling yang dideklarasikan dan diterapkan konsisten. Rancangan solusi terdiri atas tiga bagian, yaitu:

a. Dekoding payload yang menangkap seluruh field
   Aggregator mendecode payload ke struct yang memuat field wajib, dilengkapi fungsi *UnmarshalJSON* yang mendecode ulang body ke `map[string]any`. Pola yang sama dipakai untuk payload BMKG. Dengan demikian field yang tidak dikenal struct tetap tertangkap dan tidak hilang. Seluruh jalur decode memakai *json.Unmarshal* bawaan Go tanpa *DisallowUnknownFields*, sehingga keberadaan field baru tidak akan menghasilkan error.

b. Pemetaan ke format kanonik dan penerusan field tak dikenal
   Saat membangun *HazardEvent*, field yang sudah memiliki padanan kolom kanonik dikecualikan dari *attributes* agar tidak terduplikasi, sedangkan seluruh field lainnya map *attributes*. Dengan begitu deklarasi kebijakan sistem adalah: field tak dikenal diteruskan ke map *attributes*, dan tidak diabaikan. Map *attributes* disimpan pada kolom *JSONB*, sehingga record lama tanpa field baru tetap valid berdampingan dengan record baru.

c. Penanganan payload yang gagal diparse
   Payload yang rusak dianggap sebagai gangguan pada sumber. Poller mencatat error ke structured log dan menandai status sumber menjadi *DEGRADED*, lalu melanjutkan siklus polling berikutnya. Service tidak panic, dan tidak perlu di-restart maupun di-redeploy.

2.1.2. Alasan Keputusan

Berikut beberapa keputusan beserta alternatif yang dipertimbangkan dalam rancangan Problem 1.

1. Strategi decode payload

Pilihan	Struct field wajib + UnmarshalJSON kustom yang mengisi raw map
Alternatif	1. Dekode seluruhnya ke `map[string]any` 2. Struct biasa tanpa penangkap raw
Solusi	Decode penuh ke map membuat setiap field harus diambil dengan type assertion sehingga kehilangan ketepatan tipe dan membuat severity mapping rapuh. Struct biasa membuang field tak dikenal sehingga

	confidence_level akan hilang. Kombinasi keduanya memberi akses tipe-aman ke field wajib sekaligus menangkap seluruh field tak dikenal, tanpa mengubah kode saat skema berubah.

2. Kebijakan pemrosesan field tak dikenal

Pilihan	Diteruskan ke map attributes pada *HazardEvent*
Alternatif	1. Diabaikan (drop)
Solusi	Attributes memang ditujukan untuk field spesifik sumber. Mengabaikan berarti data baru hilang sehingga tidak dapat diverifikasi dan tidak dapat dimanfaatkan, dan setiap penambahan field akan selalu menuntut perubahan kode di gateway.

3. Penanganan payload yang gagal di-parse

Pilihan	Catat error ke log dan tandai sumber *DEGRADED*, lanjutkan siklus polling berikutnya
Alternatif	1. Tidak di handle 2. Buang semua record dari siklus tersebut
Solusi	Jika tidak di-handle maka akan panic, sehingga membuat seluruh service ikut berhenti termasuk ingestiion BMKG yang sehat, sehingga melanggar ketersediaan. Membuang record akan menyembunyikan masalah. Pilihan yang diambil mempertahankan availability sekaligus mengekspos masalah lewat indikator yang dapat dipantau client.

2.1.3. Batasan Yang Disadari

Berikut beberapa hal yang secara sadar tidak ditangani pada rancangan Problem 1 beserta alasannya.

1. **Field tak dikenal tidak dapat dijadikan kriteria query**

   Field baru hanya tersimpan di `attributes`, sedangkan parameter query endpoint dibatasi pada whitelist. Menjadikan `confidence_level` sebagai filter atau urutan akan menuntut penambahan kode pada API.

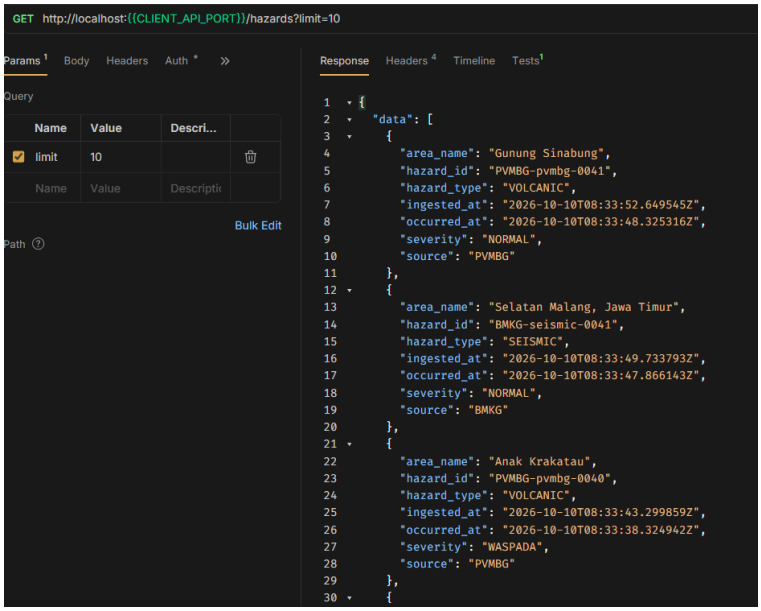
2. **Tidak ada validasi tipe maupun nilai untuk field dinamis**

`attributes` menerima nilai apa adanya dari sumber. Jika sumber mengirim tipe yang tidak terduga, data tetap tersimpan apa adanya dan konsumen bertanggung jawab menangani ketidakpastian tipe tersebut.

2.1.1. Bukti Penyelesaian

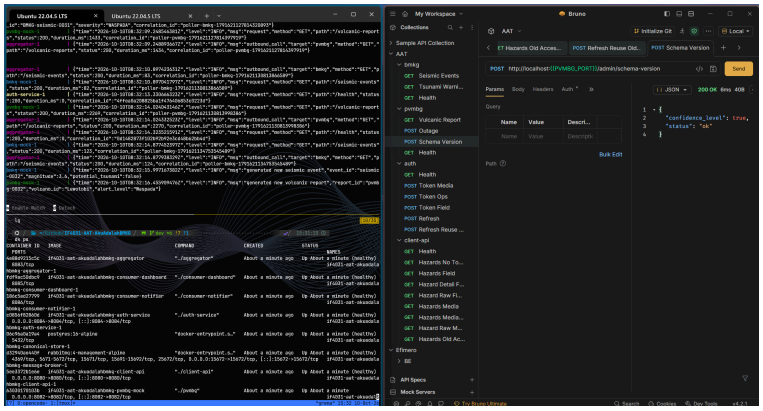
Berikut bukti terpenuhinya setiap kriteria penyelesaian Problem 3 berdasarkan pengujian yang dilakukan.

1. Aggregator menghasilkan HazardEvent yang sesuai

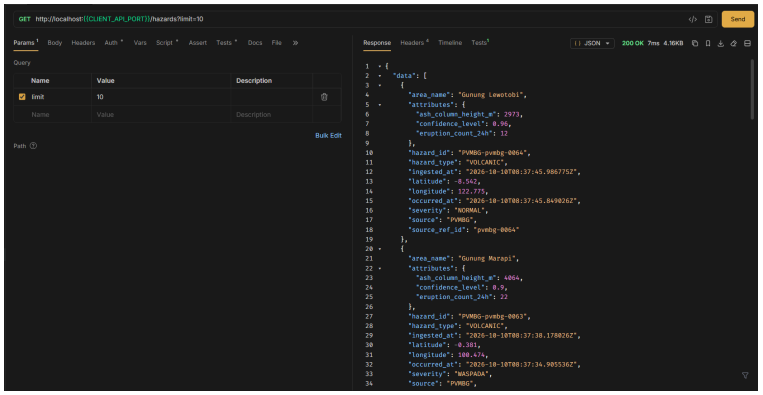
Pengujian	PVMBG menggunakan skema awal, amati Aggregator melakukan polling terhadap dua mock services
Hasil	Aggregator menghasilkan HazardEvent yang sesuai dengan aturan pemetaan untuk kedua sumber dengan PVMBG skema awal *HazardEvent dari Aggregator hasil pemetaan PVMBG dan BMKG* 

2. Perubahan skema tidak membuat sistem berhenti

Pengujian	Ubah skema PVMBG saat sistem berjalan, amati status container
Hasil	Jalani sistem, picu perubahan skema pada PVMBG melalui POST /admin/schema-version, amati dampak terhadap sistem *Tidak terjadi crash setelah perubahan skema*

3. **HazardEvent yang dihasilkan aggregator tetap valid dan perilaku konsisten dengan kebijakan**

Pengujian	Picu perubahan skema, lalu lakukan permintaan dari client terhadap data, amati data hasil keluaran.
Hasil	Aggregator menghasilkan HazardEvent yang valid, dan HazardEvent sekarang memiliki field confidence_level. confidence_level tidak diabaikan dan field dari skema lama tidak terduplikasi. *HazardEvent dari aggregator sekarang memiliki field confidence_level* 

2.1.2. Mekanisme Penanganan Field Tak Dikenal

Mekanisme yang digunakan adalah penerusan field tak dikenal ke map attributes, yang dibuat konsisten di seluruh jalur ingestiion. Alurnya sebagai berikut:

1. **Pemicu**
   Endpoint POST `/admin/schema-version` membuat PVMBG mulai menyertakan confidence_level pada laporan vulkanik berikutnya.

2. **Penangkapan**
   Poller Aggregator melakukan json.Unmarshal ke struct sumber, UnmarshalJSON mendecode ulang body ke `map[string]any` sehingga field yang tidak dikenal struct ikut tertangkap di RawFields, dan tidak dibuang.

3. **Penyaringan dan penerusan**
   Saat pemetaan kanonik, field yang sudah memiliki kolom kanonik dimasukkan ke daftar pengecualian dan tidak disalin, sedangkan seluruh field lainnya disalin ke attributes. Daftar pengecualian berlaku sama untuk ketiga tipe payload (seismic, tsunami, vulkanik), sehingga kebijakannya identik di seluruh sumber.

4. **Penyimpanan**
   attributes ditulis ke kolom JSONB dengan default '{}', sehingga record lama tanpa field baru tetap valid dan record baru tersimpan tanpa DDL migration.

5. **Penyajian**
   Client-Facing API meneruskan attributes apa adanya, sehingga client melihat field baru tanpa perubahan apa pun pada API.

6. **Toleransi**
   Decoder tidak memakai *DisallowUnknownFields*, sehingga field tak dikenal tidak menyebabkan error. Payload yang benar-benar rusak ditangani sebagai gangguan sumber (log + status DEGRADED). Consumer juga men-decode ke struct dengan attributes `map[string]any`, sehingga field tambahan tidak merusak pemrosesan.

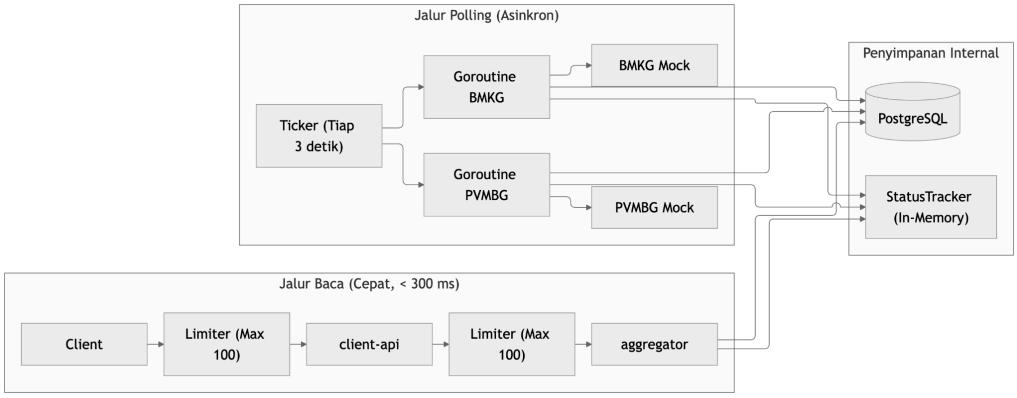
2.2. Problem 2: Concurrency, Availability, & Graceful Degradation

2.2.1. Rancangan Solusi

Permasalahan yang ada problem 2 mencakup 3 titik yaitu:

1. *Head of Line Blocking* : pemanggilan sekuensial ke BMKG dan PVMBG yang bisa menyebabkan *blocking* karena latensi yang berbeda untuk 2 pihak tersebut.
2. *Resource Exhaustion* : Tidak adanya batas jumlah *request* konkuren yang masuk ke dalam sistem. Hal ini dapat menyebabkan lonjakan kebutuhan resource pada basis data, memori, dan layanan lain yang juga dapat berakibat pada kegagalan sistem.
3. Ketiadaan *Graceful Degradation* : ketika salah satu service mengalami *outage* (contoh PVMBG *down*) maka seluruh request dari sistem ke PVMBG akan bernilai 500 / error.

Untuk mengatasi *problem* tersebut, rancangan solusi yang dibangun dibentuk seperti pada diagram berikut.



Berdasarkan diagram tersebut, terdapat beberapa solusi yang dipilih seperti:

1. Pemisahan jalur *Read* dan *Write* : *Client* hanya membaca data yang ada pada basis data dan bukan mengakses langsung PVMB maupun BMKG. Sehingga latensi tetap rendah.
2. Polling parallel dan independen : *Worker* melakukan pengambilan data dari PVMBG dan BMKG secara periodik dan terpisah tanpa kebergantungan antara satu sama lain. Sehingga keterlambatan salah satu service tidak akan menghambat proses service lain.
3. Pengendalian konkurensi dan pembatasan *resource* : semaphore berbasis *buffered channel* membatasi beban maksimum 100 permintaan konkuren. Kelebihan beban akan langsung ditolak dengan kode 429. Pembatasan juga dilakukan di sisi basis data untuk membatasi koneksi hanya sampai 100 koneksi dan menerapkan *timeout*.
4. Penerapan *Graceful Degradation* : menerapkan pelacakan status layanan PVMBG dan BMKG secara berkala. Jika layanan mati, sistem akan tetap me respons dengan kode 200 dan mengembalikan data terakhir. Ketika layanan kembali hidup, maka proses normal akan kembali bekerja.

2.2.2. Alasan Keputusan

Berikut beberapa alasan keputusan rancangan beserta alternatif yang dipertimbangkan :

1. Hubungan jalur *read* oleh *client*

Pilihan	Client hanya melakukan read ke Kanonikal Store terpisah dengan pooling
Alternatif	Permintaan client langsung diteruskan (*pass trough)* ke layanan PVMBG dan BMKG

Alasan	Pada pass trough, permintaan klien dapat terganggu bahkan gagal ketika terdapat *delay* atau *outage* pada layanan PVMBG atau BMKG

2. Eksekusi *polling* BMKG dan PVMBG

Pilihan	Menerapkan dua worker independen untuk masing-masing layanan
Alternatif	Menggunakan satu worker untuk kedua layanan
Alasan	Penggunaan dua worker akan menyebabkan kegagalan pada suatu layanan tidak akan menghambat proses lain

3. Mekanisme pengendalian beban

Pilihan	Non blocking bounded semaphore
Alternatif	Token bucket rate limiter berbasis waktu atau antrian request
Alasan	Penggunaan antrean akan menambah latensi pada sistem dan rate limiter berbasis req/detik tetap dapat mengalami penumpukan koneksi aktif

4. Penyimpanan status kesehatan

Pilihan	Struktur in-memory StatusTracker yang dilindungi sync.RWMutex
Alternatif	Menyimpan dan memperbarui status setiap siklus polling ke tabel khusus
Alasan	Menyimpan status di memori menggunakan sync.RWMutex membuat ribuan pembacaan konkuren tetap cepat tanpa membebani database, sekaligus mencegah *data race* saat diperbarui tiap 3 detik oleh *poller*

2.2.3. Batasan yang Disadari

Berikut batasan yang disadari pada problem 2:

1. Status kesehatan sumber (StatusTracker) di-*reset* saat aggregator dijalankan ulang : karena di simpan pada memori, maka status akan ter-reset ketika restart. Namun hal ini dapat teratasi karena ketika

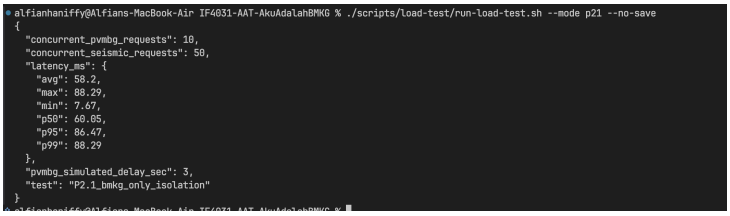
aggregator menyala, layanan ini akan melakukan pengecekan pada BMKG dan PVMBG.

2. Batas konkurensi bersifat global : pembatasan konkurensi yang dilakukan berlaku untuk setiap *request* dan tidak membedakan tipe client.
3. Data kosong jika sumber bermasalah sejak awal : jika PVMBG mati seja boot pertama dan basis data masih kosong. Maka pengambilan data oleh klien akan menghasilkan data kosong ([]) berstatus DEGRADED.

2.2.4. Bukti Penyelesaian

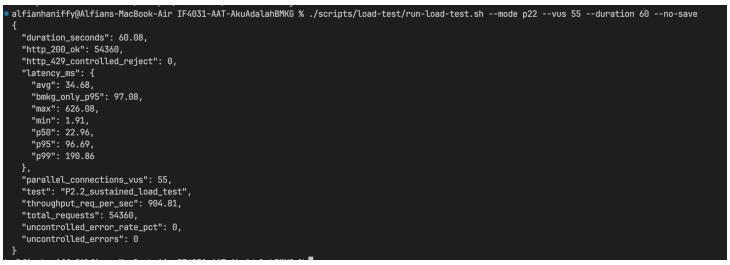
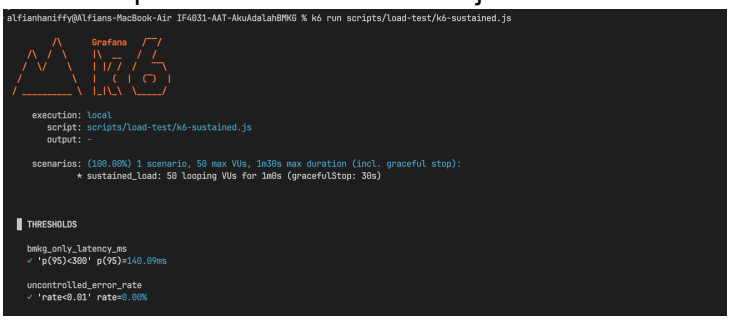
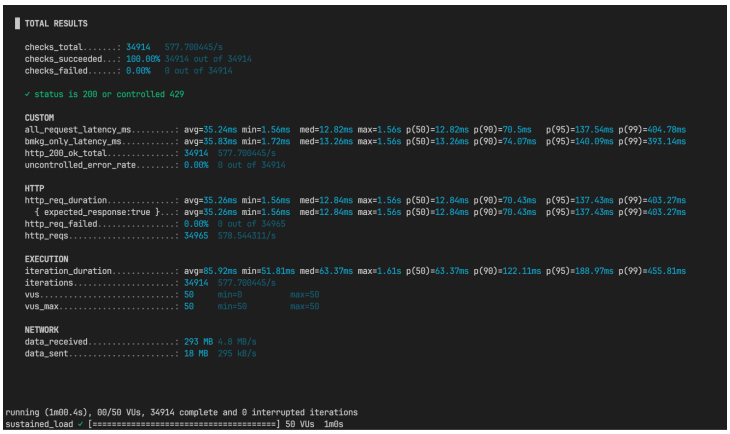
Berikut bukti penyelesaian yang dilakukan :

1. Bebas *Head-of-Line Blocking* (p95 latensi BMKG-only <300 ms , saat delay PVMBG 3 detik)

pengujian	Delay pvmbg-mock diatur secara eksplisit pada 3000 ms (3 detik) melalui POST /admin/delay. Selanjutnya dikirim 50 permintaan konkuren yang hanya membutuhkan data seismik BMKG (GET /hazards?type=SEISMIC&limit=15) secara bersamaan dengan 10 permintaan latar belakang yang mengakses data vulkanik PVMBG.
hasil	Perintah pengujian yang dieksekusi: ./scripts/load-test/run-load-test.sh --mode p21 --no-save  Penjelasan Hasil: Seluruh permintaan BMKG-only berhasil dilayani dengan status 200 OK tanpa tertahan oleh delay 3 detik milik PVMBG. Hasil pengukuran mencatat latensi min = 7,67 ms, avg = 58,2 ms, p50 = 60,05 ms, p95 = 86,47 ms, dan p99 = 88,29 ms. Pada pengujian beban berkelanjutan k6 selama 60 detik penuh.

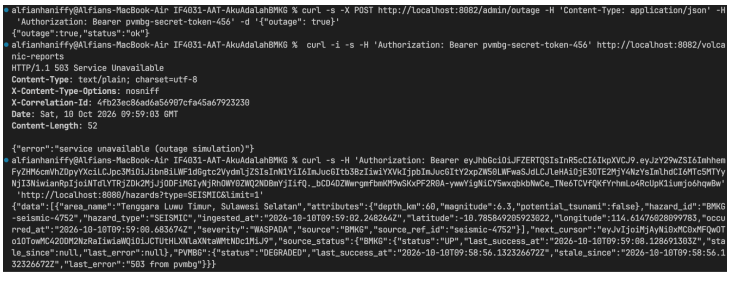
2. *Sustained Load Testing* (≥50 koneksi paralel selama ≥60 detik, *error rate* <1%)

pengujian	Dilakukan *load testing* dengan 55 koneksi paralel (55 VUs) selama 60 detik dalam pola *sustained*

	(constant-vus) ke client-api menggunakan campuran trafik 50% SEISMIC (BMKG-only), 30% VOLCANIC (PVMBG), dan 20% gabungan (GET /hazards?limit=20).
hasil	**Perintah pengujian yang dieksekusi:** ./scripts/load-test/run-load-test.sh --mode p22 --vus 55 --duration 60 --no-save  atau dijalankan langsung menggunakan Grafana k6: k6 run scripts/load-test/k6-sustained.js   **Penjelasan Hasil:** Selama 60 detik pengujian pada kedua *engine*, tidak ada satu pun *service* yang mengalami *crash* dan **uncontrolled error rate tercatat 0,00%** (jauh di bawah batas 1%). Rincian metrik *throughput*, latensi p50/p95/p99, dan *error rate* disajikan pada tabel hasil di

	bawah.

3. Simulasi *Outage* PVMBG, *Graceful Degradation*, dan Pemulihan Otomatis

pengujian	Pengujian dilakukan dengan mensimulasikan kegagalan total pada layanan PVMBG melalui langkah berurutan: mengaktifkan kondisi *outage* pada PVMBG (POST /admin/outage), mengamati penolakan HTTP 503 dari PVMBG, memverifikasi ketersediaan layanan seismik dan degradasi terkendali pada layanan vulkanik dari client-api, lalu mematikan kondisi *outage* dan memverifikasi pemulihan otomatis source_status tanpa *restart*.
hasil	**Perintah pengujian yang dieksekusi secara berurutan:** 1. curl -s -X POST http://localhost:8082/admin/outage -H 'Content-Type: application/json' -H 'Authorization: Bearer <pvmbg-token>' -d '{"outage": true}' 2. curl -i -s -H 'Authorization: Bearer <pvmbg-token>' http://localhost:8082/volcanic-reports 3. curl -s -H 'Authorization: Bearer <ops-token>' 'http://localhost:8080/hazards?type=SEISMIC&limit=1' 4. curl -s -H 'Authorization: Bearer <ops-token>' 'http://localhost:8080/hazards?type=VOLCANIC&limit=1' 5. curl -s -X POST http://localhost:8082/admin/outage -H 'Content-Type: application/json' -H 'Authorization: Bearer <pvmbg-token>' -d '{"outage": false}' 6. curl -s -H 'Authorization: Bearer <ops-token>' 'http://localhost:8080/hazards?limit=1' | jq '.source_status' 



Penjelasan Hasil:
1) Selama PVMBG *outage*, permintaan data seismik tetap dilayani normal (200 OK, BMKG.status = "UP").
2) Permintaan data vulkanik tetap dijawab cepat dengan status **200 OK** (bukan 500 dan tidak menggantung) berisi data gunung api terakhir dari Canonical Store (PVMBG-pvmbg-4751), disertai penanda degradasi eksplisit pada source_status.PVMBG: "status": "DEGRADED", "last_success_at": "2026-10-10T09:58:56.132326672Z", "stale_since": "2026-10-10T09:58:56.132326672Z", dan "last_error": "503 from pvmbg".
3) Setelah *outage* dimatikan, siklus *polling* berikutnya otomatis berhasil dan mengembalikan source_status.PVMBG.status menjadi "UP" serta stale_since: null tanpa *restart*.

2.2.5. Model Konkurensi dan Konfigurasi *Load Testing*

2.2.6. Pencegahan tiga titik kegagalan

1. Head-of-Line Blocking: *Poller* memisahkan pengambilan data BMKG dan PVMBG ke dua goroutine independen, sehingga PVMBG yang lambat tidak menghambat BMKG. Di sisi baca, *client* membaca langsung dari PostgreSQL tanpa memicu panggilan jaringan ke pihak ketiga.
2. Resource Exhaustion: Pembatasan beban menggunakan *buffered channel* (kapasitas 100) secara *non-blocking*. Permintaan berlebih langsung ditolak dengan status 429 Too Many Requests. Basis data turut diproteksi melalui *connection pool* (maksimal 100 koneksi terbuka).
3. Kegagalan Total (Graceful Degradation): Gangguan pada sumber eksternal hanya mengubah status di StatusTracker menjadi DEGRADED. Layanan tetap menyajikan data historis terakhir dari PostgreSQL dengan status 200 OK.

2.2.7. Alat dan parameter pengujian beban

Parameter	Nilai	Keterangan
Target Endpoint	http://localhost:8080/hazards	Melalui pipeline penuh client-api
Identitas Uji	bnpb-ops	Akses data mentah lengkap (all fields)
Refresh Token	Tiap 40s (k6) / 25s (Go)	Mencegah kedaluwarsa token JWT (TTL 60s)
Beban Paralel (VUs)	55 VUs	Memenuhi target pengujian (≥ 50 koneksi)
Durasi	60 detik (sustained)	Memenuhi target durasi (≥ 60 detik)
Distribusi Trafik	50% Seismik, 30% Vulkanik, 20% Gabungan	Mensimulasikan pola trafik nyata
Jeda Iterasi	25–50 ms	Menghasilkan ~568–763 request/detik
Ambang Batas Lolos	p(95) < 300 ms & error rate < 1%	Standar kelulusan uji latensi dan stabilitas

2.3. Problem 3: Service-to-Service Authentication & Trust

2.3.1. Rancangan Solusi

Permasalahan utama pada Problem 3 adalah tidak adanya pemisahan domain kepercayaan (*domain of trust*), dimana satu kredensial yang bocor dapat berdampak ke seluruh sistem. Untuk mengatasinya, sistem dibagi menjadi tiga domain kepercayaan yang saling independen. Setiap domain memiliki kredensial, pemegang, dan pihak pemverifikasi yang berbeda seperti ditunjukkan pada tabel berikut.

Domain	Kredensial	Pemegang	Pemverifikasi
BNPB ke BMKG	API key pada header X-BMKG-Key	Aggregator	BMKG Mock
BNPB ke PVMBG	Token pada header Authorization: Bearer	Aggregator	PVMBG Mock
Client ke BNPB	Access token JWT dari Auth Service	Client downstream	Client-Facing API

Berdasarkan pembagian tersebut, rancangan solusi Problem 3 terdiri atas empat bagian, yaitu:

a. Pengamanan kredensial instansi sumber

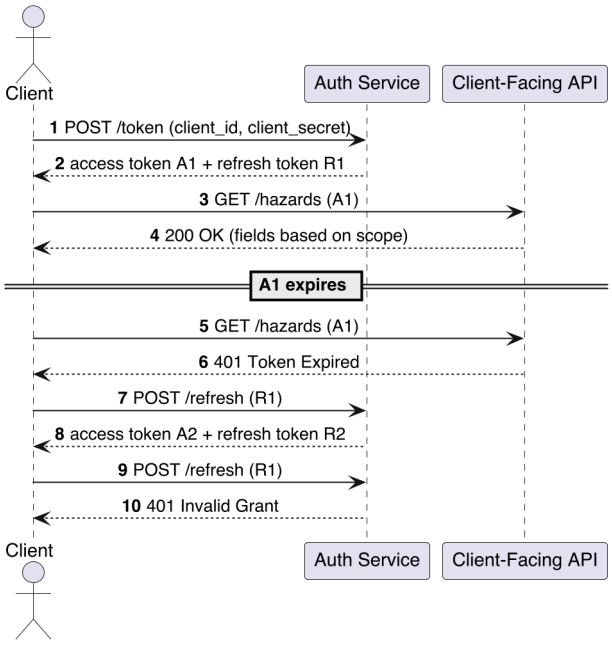
BMKG dan PVMBG menggunakan kredensial dengan format dan nilai yang berbeda. Setiap mock hanya menerima kredensial miliknya sendiri dan menolak kredensial lain dengan status `401 Unauthorized`. Kedua kredensial tersebut hanya dipegang oleh Aggregator sebagai satu-satunya komponen yang berkomunikasi dengan instansi sumber. Client-Facing API dan client downstream tidak pernah memiliki akses terhadap kredensial ini.

b. Penerbitan token untuk client downstream

Client melakukan autentikasi ke Auth Service menggunakan `client_id` dan `client_secret` miliknya untuk memperoleh token. Media/Pers, Tim Lapangan, dan BNPB Pusat/Internal Ops diimplementasikan sebagai tiga identitas terpisah, masing-masing dengan kredensial sendiri. Scope setiap client ditetapkan oleh konfigurasi di sisi server dan bukan dari permintaan client. Media memperoleh scope `hazards:read:summary`, sedangkan Tim Lapangan dan BNPB Ops memperoleh scope `hazards:read:raw`. Setelah autentikasi berhasil, Auth Service menerbitkan dua jenis token, yaitu:

1. *Access Token*, berupa JSON Web Token (JWT) yang ditandatangani secara asimetris menggunakan algoritma EdDSA. Token memuat scope client dan waktu kedaluwarsa. Masa berlakunya 60 detik secara default dan dapat dikonfigurasi.
2. *Refresh token*, berupa string acak yang disimpan di Auth Store (Redis) dalam bentuk hash.

Refresh token bersifat sekali pakai. Setiap proses *refresh* menerbitkan pasangan token baru dan membatalkan token lama, sehingga client dapat melanjutkan sesi tanpa login ulang. Apabila *refresh token* yang sudah terpakai digunakan kembali, Auth Service mencabut seluruh *token family* yang terkait. Berikut alur lengkap proses proses tersebut.



c. Penegakan hak akses di Client-Facing API

Client-Facing API memverifikasi JWT secara lokal menggunakan *public key* yang dibaca dari environment variable tanpa memanggil Auth Service. Verifikasi mencakup keabsahan tanda tangan, algoritma, penerbit dan penerima token, serta waktu kedaluwarsa. Permintaan tanpa token, dengan token tidak valid, atau dengan token kedaluwarsa ditolak dengan status `401 Unauthorized`.

Setelah token terverifikasi, respons disesuaikan dengan scope client. Client dengan scope *summary* (Media) hanya menerima field berklasifikasi Ringkasan. Field Mentah akan dihapus di sisi server sebelum respons dikirim, sehingga data tersebut tidak pernah sampai ke client. Apabila Media secara eksplisit meminta field Mentah, permintaan tersebut ditolak dengan status `403 Forbidden`. Client dengan scope *raw* (Tim Lapangan dan BNPB Ops) dapat menerima seluruh field.

d. Pengelolaan Kredensial

Seluruh kredensial dibaca dari berkas `.env` yang dikecualikan dari version control dan setiap service menyertakan berkas `.env.example` berisi daftar variabel dengan nilai placeholder. Service akan menolak dijalankan apabila terdapat variabel wajib yang kosong, sehingga tidak ada nilai rahasia *default* yang digunakan tanpa disadari.

2.3.2. Alasan Keputusan

Berikut beberapa keputusan beserta alternatif yang dipertimbangkan dalam rancanan Problem 3.

1. **Algoritma tanda tangan *access token***

Pilihan	EdDSA
Alternatif	1. HMAC 2. RSA
Alasan	HMAC bersifat simetris, yaitu menggunakan satu secret untuk menandatangani sekaligus memverifikasi token, sehingga Client-Facing API juga dapat menerbitkan token. RSA dan EdDSA bersifat asimetris, sehingga Client-Facing API hanya dapat memverifikasi token. EdDSA dipilih karena ukuran kuncinya lebih kecil dan proses tanda tangannya lebih cepat dibandingkan RSA dengan tingkat keamanan yang setara.

2. **Distribusi *public key***

Pilihan	*Environment variable*
Alternatif	Endpoint JWKS milik Auth Service
Alasan	JWKS membuat Client-Facing API bergantung pada Auth Service, sedangkan *environment variable* memungkinkan verifikasi tetap berjalan meskipun Auth Service tidak aktif. Konsekuensinya adalah penggantian kunci harus dilakukan secara manual dengan memperbarui konfigurasi dan menjalankan ulang kedua service.

3. **Penyimpanan *refresh token***

Pilihan	Redis
Alternatif	1. Penyimpanan *in-memory* di Auth Service 2. PostgreSQL
Alasan	Penyimpanan *in-memory* membuat seluruh refresh token hilang ketika Auth Service dijalankan ulang, sehingga client harus login ulang. PostgreSQL tidak memiliki mekanisme kedaluwarsa bawaan, sehingga dibutuhkan kolom tambahan dan proses pembersihan berkala. Redis mendukung kedaluwarsa data secara bawaan dan datanya tetap bertahan saat Auth Service dijalankan ulang.

4. **Pemisahan akses berdasarkan scope**

Pilihan	Satu endpoint yang sama untuk seluruh client dengan response yang disesuaikan berdasarkan scope token
Alternatif	Endpoint terpisah untuk setiap jenis client

Alasan	Endpoint terpisah menggandakan logika yang sama hanya untuk membedakan field yang dikirim, dan setiap penambahan jenis client akan menambah endpoint baru. Sebaliknya, dengan satu endpoint, aturan akses cukup ditentukan oleh scope di dalam token.

2.3.3. Batasan Yang Disadari

Berikut beberapa hal yang secara sadar tidak ditangani pada rancangan Problem 3 beserta alasannya.

1. ***Access token* lama tetap berlaku hingga kedaluwarsa**
   Setelah *refresh*, *access token* lama masih diterima hingga waktu kedaluwarsanya karena Client-Facing API tidak memeriksa status token ke Auth Service. Hal ini disengaja agar Client-Facing API tetap dapat berjalan tanpa bergantung pada Auth Service.

2. **Penggantian kunci dilakukan secara manual**
   Penggantian pasangan kunci JWT dilakukan dengan memperbarui konfigurasi lalu menjalankan ulang Auth Service dan Client-Facing API. Rotasi otomatis tidak diterapkan karena mengharuskan Client-Facing API mengambil kunci dari Auth Service, sehingga keduanya tidak lagi independen. Selain itu, spesifikasi juga tidak mensyaratkan rotasi kunci.

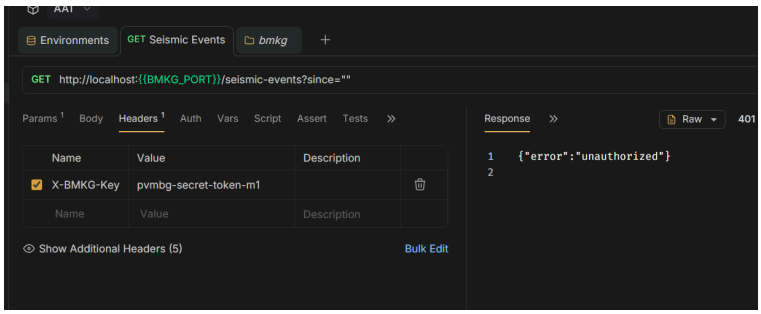
3. **Sebagian komunikasi internal tidak diautentikasi**
   Panggilan Client-Facing API ke Aggregator dan koneksi Auth Service ke Auth Store tidak menggunakan kredensial, tetapi hanya dilindungi oleh isolasi jaringan Docker. Autentikasi tambahan tidak diterapkan karena keduanya tidak dapat diakses dari luar jaringan tersebut.

2.3.4. Bukti Penyelesaian

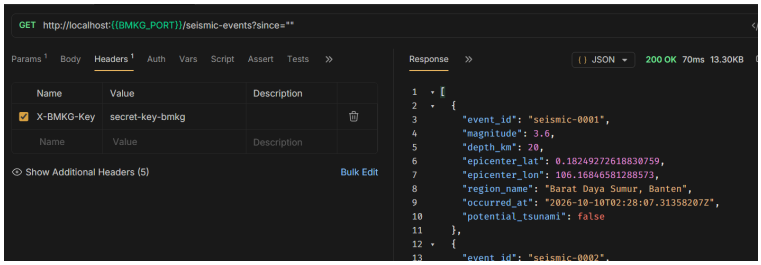
Berikut bukti terpenuhinya setiap kriteria penyelesaian Problem 3 berdasarkan pengujian yang dilakukan.

1. **Kredensial instansi tidak berlaku lintas instansi**

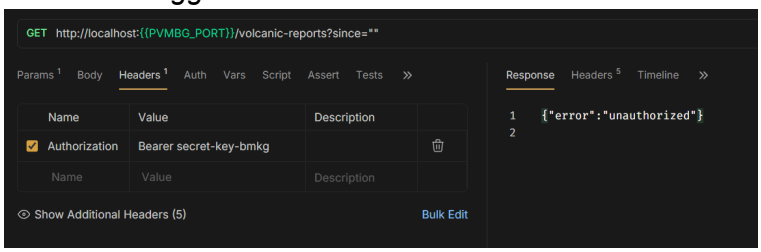
Pengujian	API key BMKG digunakan untuk mengakses PVMBG, dan token PVMBG digunakan untuk mengakses BMKG.
Hasil	Kedua permintaan ditolak dengan status `401`, sedangkan permintaan dengan kredensial yang sesuai berhasil dengan status `200`. BMKG_API_KEY=secret-key-bmkg PVMBG_TOKEN=pvmbg-secret-token-m1 *BMKG menggunakan kredensial PVMBG*



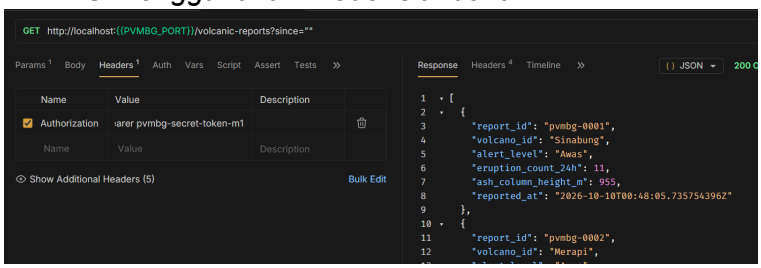
BMKG menggunakan kredensial benar



PVMBG menggunakan kredensial BMKG

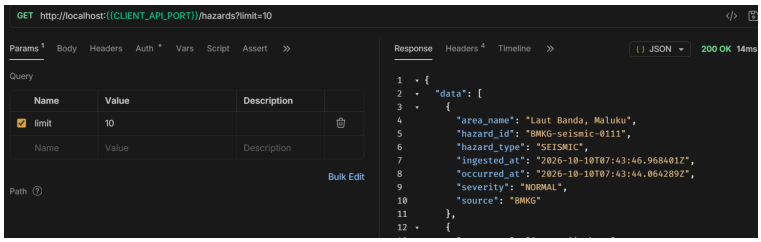
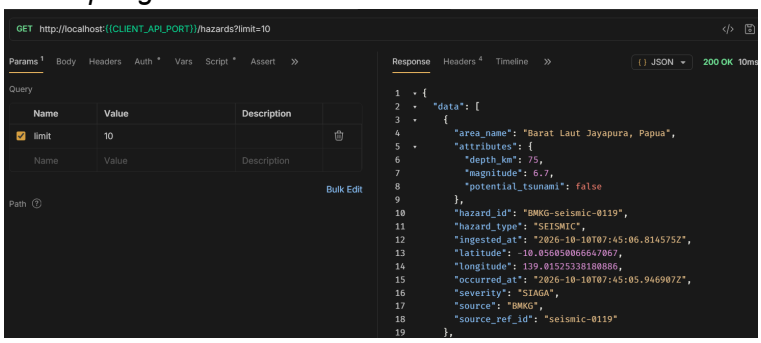
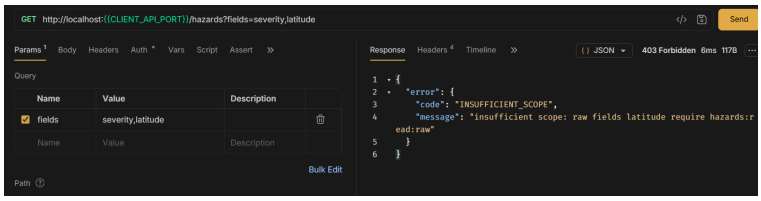


PVMBG menggunakan kredensial benar

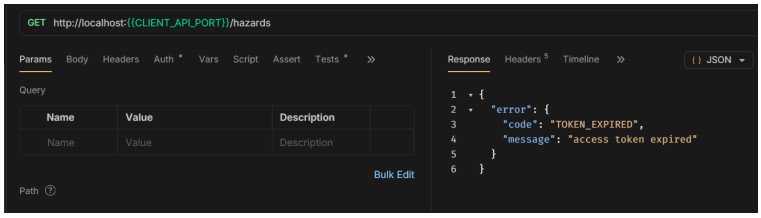


2. Media hanya menerima field Ringkasan

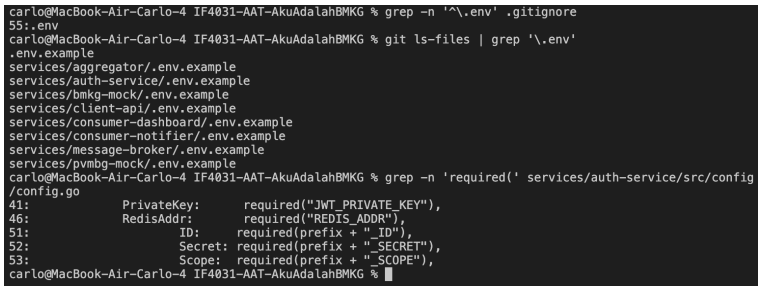
Pengujian	Media dan Tim Lapangan mengirim permintaan yang sama ke `GET /hazards`. Media kemudian meminta field Mentah secara eksplisit.
Hasil	Media hanya menerima field Ringkasan, sedangkan Tim Lapangan menerima seluruh field. Permintaan eksplisit

	Media terhadap field Mentah ditolak dengan status `403`. *Media*  *Tim Lapangan*  *Media meminta RAW* 

3. **Kedaluwarsa dan refresh token**

Pengujian	Token di-refresh, dan client mencoba meminta data dengan token lama
Hasil	Permintaan dengan token kedaluwarsa ditolak dengan kode 401. *Token lama ditolak* 

4. **Tidak ada kredensial ter-*hardcode***

Pengujian	Repositori diperiksa untuk memastikan berkas `.env` tidak ter-*commit*, kredensial dibaca dari *environment variable*, dan berkas `.env.example` tersedia di setiap service.
Hasil	Berkas `.env` dikecualikan oleh `.gitignore` dan tidak ditemukan di repositori. Seluruh kredensial dibaca dari environment variable, dan setiap service menyediakan berkas `.env.example`. 

2.3.5. Mekanisme Autentikasi dan Analisis Celah Keamanan

Mekanisme yang digunakan adalah pemisahan kredensial berdasarkan domain kepercayaan. Aggregator mengakses BMKG dan PVMBG menggunakan kredensial yang berbeda untuk masing-masing instansi. Client downstream menggunakan access token JWT yang diterbitkan Auth Service. Token tersebut memuat scope yang ditetapkan server, berlaku singkat, dan dapat diperpanjang melalui *refresh token* sekali pakai.

Mekanisme ini tidak membuka celah baru karena tidak ada satu kredensial pun yang berlaku di lebih dari satu domain, sehingga kebocoran pada satu komponen hanya berdampak pada domainnya sendiri. Berikut beberapa skenario kebocoran beserta dampaknya.

1. Kebocoran *secret* Media hanya memberikan akses ke field Ringkasan karena scope ditetapkan oleh server dan token yang diubah isinya akan ditolak.
2. Client-Facing API yang diambil alih penyerang tidak dapat digunakan untuk menerbitkan token karena hanya memegang *public key*.
3. Kebocoran kredensial BMKG atau PVMBG tidak memberikan akses ke instansi lain.
4. *Refresh token* yang dicuri hanya dapat digunakan sekali dan penggunaan ganda akan membatalkan seluruh sesi terkait.
5. Kebocoran isi Auth Store tidak memberikan *refresh token* yang dapat digunakan karena yang disimpan hanya nilai *hash*-nya.

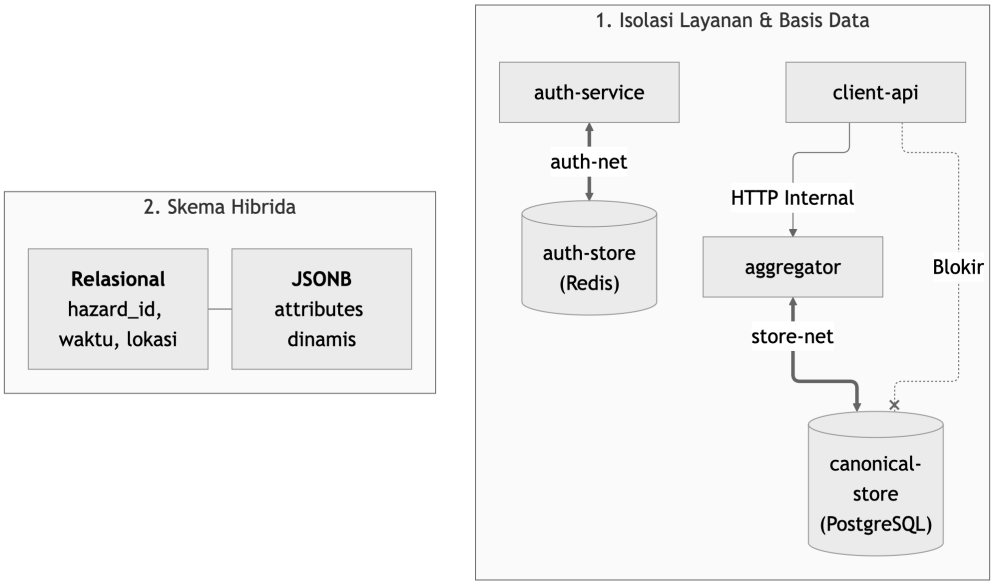
2.4. Problem 4: Arsitektur Microservice & Flexible Storage

2.4.1. Rancangan Solusi

Permasalahan utama pada problem 4 terdiri dari:

1. **Keterikatan unit *deployment* (*monolithic deployment coupling*)**: Ketika logika *polling*, pemetaan kanonik, autentikasi, dan penyajian API digabung dalam satu layanan monolitik, perubahan kecil pada satu modul (misalnya autentikasi) memaksa *build* dan *redeploy* seluruh sistem.
2. **Kekakuan skema penyimpanan (*rigid relational storage*)**: Menyimpan seluruh atribut bencana pada kolom-kolom tabel relasional yang kaku memaksa eksekusi migrasi skema (ALTER TABLE) dan koordinasi *deployment* setiap kali instansi sumber menambahkan *field* baru seperti

Untuk menyelesaikan permasalahan tersebut, rancangan solusi digambarkan seperti diagram berikut:



Berdasarkan diagram tersebut ada beberapa rancangan yang dipilih:

1. Dekomposisi Microservice Mandiri
    a. Setiap modul (aggregator, client-api, auth-service, mock) dibangun dalam container terpisah dengan konfigurasi .env dan *dependency* masing-masing, sehingga dapat di-*deploy* atau di-*restart* secara independen.
    b. Hubungan antar-layanan bersifat *loosely coupled*. Jika auth-service mati, client-api tetap dapat melayani GET /hazards menggunakan verifikasi lokal JWT Ed25519. Jika aggregator mati sementara, client-api merespons secara terkontrol dengan galat 503 Service Unavailable tanpa mengalami *crash*.
2. Penyimpanan fleksibel menggunakan JSONB

   a. 9 kolom kanonik inti disimpan pada kolom bertipe relasional tetap agar filter pencarian dan *keyset pagination*berbasis indeks B-Tree (occurred_at, hazard_id) tetap optimal.
   b. Atribut dinamis disimpan dalam kolom attributes JSONB. Ketika ada *field* baru dari sumber (seperti confidence_level), data langsung diserialisasi ke dalam JSONB secara otomatis tanpa perlu ALTER TABLE atau *downtime*.
3. Isolasi ekslusif Canonical Store
   a. canonical-store ditempatkan di jaringan terisolasi store-net (internal: true) tanpa publikasi port keluar.
   b. Hanya aggregator yang memiliki akses jaringan dan kredensial DATABASE_URL. Layanan client-api wajib mengakses data melalui endpoint privat GET /internal/hazards.

2.4.2. Alasan Keputusan

1. Pemisahan Aggregator dan Client Facing API ke dalam 2 service berbeda

Pilihan	Memisahkan aggregator dan client API
Alternatif	Menggabungkan aggregator dan client api menjadi satu
Alasan	Menjaga prinsip single responsibility. Perubahan aturan pada client tidak akan mengganggu aggregator

2. Teknologi penyimpanan untuk Canonical Store

Pilihan	Basisdata relasional dengan JSONB
Alternatif	Basis data non relasional / KV store (mongoDB/redis)
Alasan	Menjamin integritas ACID dan performa index filter multi kolom namun tetap dapat menampung atribut fleksibel dengan JSONB

3. Mekanisme paginasi pada API Internal Aggregator dan Canonical Store

Pilihan	Keyset pagination menggunakan cursor
Alternatif	Offset pagination
Alasan	Menghindari data bergeser atau duplikat ketika data datang terus menerus namun tetap memberi kueri yang efisien

4. Penegakan kepemilikan *storage* di tingkat jaringan Docker

Pilihan	Jaringan privat pada docker network (auth-net , store-net)
Alternatif	Satu jaringan bersama
Alasan	Mencegah akses langsung ke basis data melalu layanan yang tidak berhak

2.4.3. Batasan yang Disadari

Berikut beberapa batasan yang disadari pada implementasi :

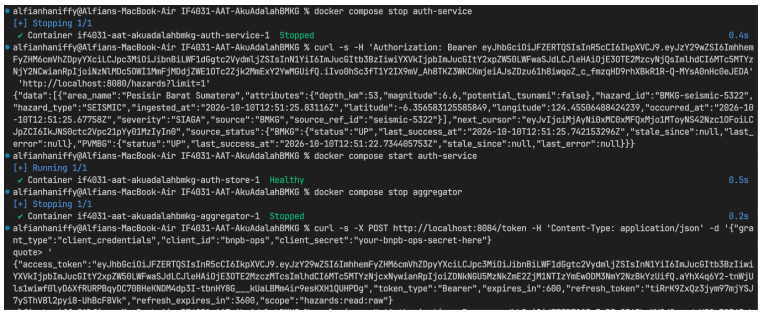
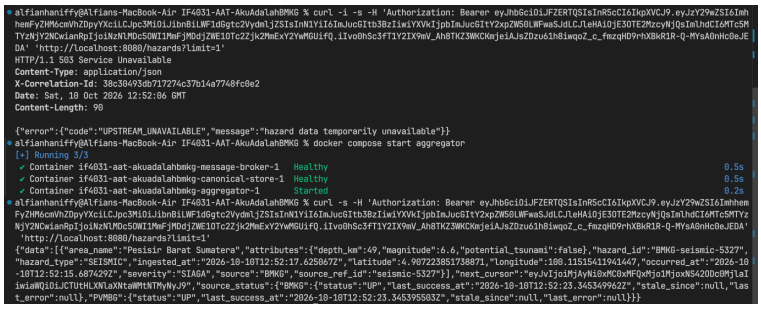
1. Ketidaktersediaan aggregator akan mengakibatkan gagal baca pada client api : client api tidak diizinkan mengakses canonical store secara langsung dan harus melalui agregator sehingga ketika aggregator mengalami down maka client api tidak dapat memberi jawaban kepada downstream client
2. Tidak ada validasi tipe data di tingkat ddl database untuk isi kolom atribute dinamik (JSONB) : penggunaan jsonb memungkinkan database untuk menyimpan segala *field* data variatif pada kolom ini, namun hal ini juga berpengaruh pada basis data yang tidak dapat melakukan validasi tertentu pada field-field dinamik yang mungkin muncul
3. Setiap service dan penyimpanan berjalan dalam single instance : belum ada penerapan replica atau clustering pada sistem ini. Sehingga setiap layanan hanya memiliki satu container yang berjalan.

2.4.4. Bukti Penyelesaian

Berikut adalah bukti penyelesaian terhadap problem 4:

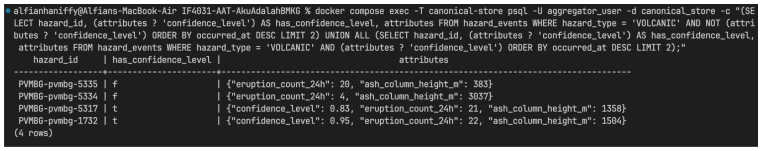
1. Independensi eksekusi container microservices

Pengujian	Pengujian dilakukan dengan menghentikan salah satu container secara mandiri (auth-service) untuk membuktikan client-api tetap melayani pembacaan dengan token aktif, kemudian menghentikan aggregator untuk membuktikan auth-service tetap menerbitkan token sementara client-api merespons 503 UPSTREAM_UNAVAILABLE secara terkontrol, dan akhirnya menyalakan kembali container terkait tanpa me-*restart* container lain.
Hasil	**Perintah pengujian yang dieksekusi secara berurutan:** 1. docker compose stop auth-service 2. curl -s -H 'Authorization: Bearer <ops-token>' 'http://localhost:8080/hazards?limit=1' 3. docker compose start auth-service 4. docker compose stop aggregator 5. curl -s -X POST http://localhost:8084/token -H

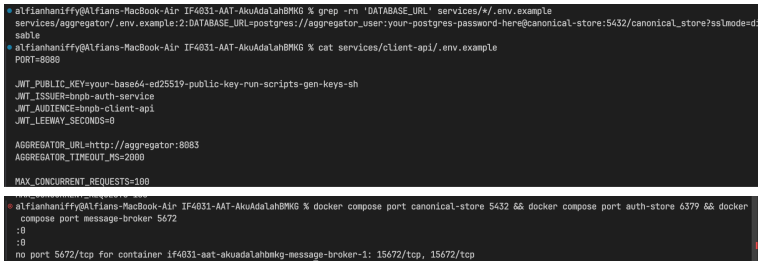
	'Content-Type: application/json' -d '{"grant_type":"client_credentials","client_id":"bnpb-ops","client_secret":"<ops-secret>"}' 6. curl -i -s -H 'Authorization: Bearer <ops-token>' 'http://localhost:8080/hazards?limit=1' 7. docker compose start aggregator 8. curl -s -H 'Authorization: Bearer <ops-token>' 'http://localhost:8080/hazards?limit=1'   **Penjelasan Hasil:** 1) Saat auth-service dihentikan, client-api tetap merespons status 200 OK dengan data bencana karena verifikasi JWT dilakukan secara mandiri menggunakan public key Ed25519 lokal. 2) Saat aggregator dihentikan, auth-service tetap berhasil menerbitkan token (200 OK), dan client-api merespons secara terkontrol dengan 503 Service Unavailable (UPSTREAM_UNAVAILABLE). Begitu aggregator dinyalakan kembali, client-api otomatis kembali merespons 200 OK tanpa me-*restart* container lain.

2. Penyimpanan fleksible (JSONB) tanpa migrasi skema saat muncul confidence_level

Pengujian	Sistem dijalankan dengan skema awal PVMBG (tanpa confidence_level), lalu perubahan skema dipicu melalui POST /admin/schema-version ({"enabled": true}). Selanjutnya isi tabel hazard_events di canonical-store diperiksa langsung menggunakan kueri SQL PostgreSQL (attributes ? 'confidence_level') dan dibaca melalui GET

	/hazards?type=VOLCANIC.
Hasil	**Perintah pengujian yang dieksekusi:** 1. Kueri langsung ke Canonical Store (PostgreSQL): docker compose exec -T canonical-store psql -U aggregator_user -d canonical_store -c "(SELECT hazard_id, (attributes ? 'confidence_level') AS has_confidence_level, attributes FROM hazard_events WHERE hazard_type = 'VOLCANIC' AND NOT (attributes ? 'confidence_level') ORDER BY occurred_at DESC LIMIT 2) UNION ALL (SELECT hazard_id, (attributes ? 'confidence_level') AS has_confidence_level, attributes FROM hazard_events WHERE hazard_type = 'VOLCANIC' AND (attributes ? 'confidence_level') ORDER BY occurred_at DESC LIMIT 2);"  2. Pembacaan via Client-Facing API: curl -s -H 'Authorization: Bearer <ops-token>' 'http://localhost:8080/hazards?type=VOLCANIC&limit=100' | jq '{old_schema_sample: ([.data[]? | select(.attributes.confidence_level == null)][0] // null), new_schema_sample: ([.data[]? | select(.attributes.confidence_level != null)][0] // null)}'  **Penjelasan Hasil:** Tanpa adanya perintah ALTER TABLE atau migrasi skema apa pun, record lama (has_confidence_level = f) dan record baru (has_confidence_level = t) tersimpan berdampingan di dalam kolom attributes (JSONB) pada tabel hazard_events, dan keduanya dapat dibaca secara utuh melalui API.

3. Kepemilikan eksklusif kanonikal store oleh aggregator

Pengujian	Pengujian dilakukan dengan memeriksa konfigurasi variabel DATABASE_URL pada seluruh direktori services/*/, memeriksa konfigurasi AGGREGATOR_URL pada services/client-api, serta memeriksa isolasi port database melalui docker compose port.
Hasil	**Perintah pengujian yang dieksekusi:** 1. grep -rn 'DATABASE_URL' services/*/.env.example 2. cat services/client-api/.env.example 3. docker compose port canonical-store 5432 && docker compose port auth-store 6379 && docker compose port message-broker 5672  **Penjelasan Hasil:** Pemeriksaan membuktikan hanya services/aggregator yang memiliki konfigurasi koneksi langsung ke Canonical Store (DATABASE_URL), sedangkan services/client-api hanya mengakses AGGREGATOR_URL=http://aggregator:8083 tanpa driver database langsung. Selain itu, port 5432 pada canonical-store terbukti tidak dipublikasikan ke host (not published) dan diisolasi di dalam network internal store-net.

2.4.5. Alasan Penentuan Batas *Service* dan Perbandingan Alternatif Penyimpanan

2.4.6. Alasan penentuan batas layanan

Layanan	Tanggung jawab utama	Alasan
aggregator	Ingest data, pemetaan kanonik, publish data	Menyatukan polling BMKG dan PVMBG, serta memungkinkan melakukan transaksi outbox (publish)
client-api	Verifikasi jwt, pembatasan akses, filter field, load control	Menerapkan prinsip least privilege dan ditempatkan pada lapisan terluar

auth-service	Autentikasi client, penerbitan token	Mengisolasi manajemen autentikasi dan kunci privat
consumer	Konsumsi pesan rabbitMQ	Memisahkan kapabilitas downstream agar kegagalan pada suatu konsumer tidak mengganngu konsumer lain

2.4.7. Perbandingan alternatif penyimpanan

Kriteria	PostgreSQL + JSONB	MongoDB	Redis
Perubahan skema	Sangat baik, penggunaan JSONB memungkinkan perubahan field	Sangat baik, secara bawaan sudah schemaless	Cukup, bisa via redis json
Integritas field utama	Sangat baik, dapat menjaga integritas field utama	Lemah, perlu validasi tambahan	Tidak ada
Key set pagination	Baik, tuple (occured_at dan hazard_id) dapat di index	Cukup, perlu compound index	kurang , butuh sorted set
ACID & outbox	Sangat baik, transaksi bersifat atomik untuk operasi insert dan lainnya	Cukup, pipeline update lebih kompleks untuk cek perubahan data	Rumit, memerlukan skrip custom

2.4.8. Konsekuensi pilihan terpilih

Berikut adalah konsekuensi menggunakan postgres + jsonB:

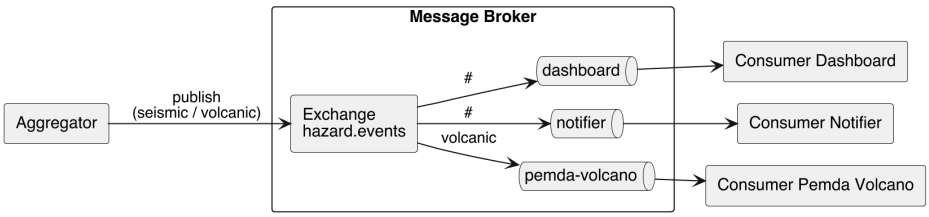
1. Keuntungan: Mendapat fleksibilitas dokumen tanpa kehilangan integritas relasional, filter multi-kolom tetap cepat, dan tabel sekaligus berfungsi sebagai *transactional outbox*.
2. Trade-off: Kode Go perlu melakukan json.Marshal/Unmarshal manual untuk kolom attributes, serta validasi tipe data di dalam JSONB didelegasikan ke level aplikasi

2.5. Problem 5: Fan-out Event Distribution/Pub-sub

2.5.1. Rancangan Solusi

Permasalahan utama pada Problem 5 adalah keterikatan langsung antara *producer* dan *consumer*. Aggregator harus mengenal dan memanggil setiap consumer secara sinkron sehingga consumer yang lambat atau gagal akan ikut menghambat yang lain. Untuk mengatasinya, Aggregator mempublikasikan event ke *message broker* RabbitMQ, bukan mengirimkannya langsung ke consumer. Setiap consumer kemudian menerima event dari broker tanpa perlu dikenal oleh Aggregator.

Berikut rancangan topologi broker beserta hubungan antara Aggregator, exchange, queue, dan consumer.



a. Topologi broker

Aggregator mempublikasikan setiap `HazardEvent` ke *exchange* bertipe *topic* bernama `hazard.events`, dengan *routing key* `seismic` atau `volcanic` sesuai jenis bahayanya. Setiap consumer memiliki queue sendiri yang diikat ke *exchange* tersebut dan broker menyalin setiap event ke seluruh queue yang sesuai. Consumer Dashboard dan Consumer Notifier berlangganan seluruh event dengan *routing key* `#`, sedangkan Consumer Pemda Volcano hanya berlangganan event vulkanik.

b. Publikasi event oleh Aggregator

Aggregator hanya mempublikasikan `HazardEvent` yang baru atau berubah setelah disimpan ke Canonical Store. Pesan dikirim sebagai pesan *persistent* dan Aggregator menunggu konfirmasi penerimaan dari broker. Setiap pesan membawa `message_id` berupa gabungan `hazard_id` dan `severity` sebagai penanda unik. Aggregator hanya mengenal *exchange* dan tidak mengetahui queue maupun consumer yang berlangganan.

c. Penerimaan event oleh consumer

Setiap consumer mendeklarasikan queue miliknya sendiri sebagai queue *durable*, sehingga pesan tetap tersimpan di broker ketika consumer tidak berjalan. Consumer mengonfirmasi pesan secara manual setelah pesan

selesai diproses sehingga pesan yang belum selesai diproses akan dikirim ulang oleh broker. Consumer Notifier juga mencatat `message_id` yang sudah diproses agar pesan yang diterima lebih dari sekali tidak menghasilkan notifikasi ganda.

2.5.2. Alasan Keputusan

Berikut beberapa keputusan beserta alternatif yang dipertimbangkan dalam rancanan Problem 5.

1. Message broker

Pilihan	RabbitMQ
Alternatif	1. NATS JetStream 2. Apache Kafka
Alasan	Kafka dinilai terlalu kompleks untuk skala sistem ini karena memerlukan pengelolaan partisi dan *offset*. NATS JetStream lebih ringan dan mendukung fitur *replay*, tetapi fitur tersebut tidak dibutuhkan dalam spesifikasi. RabbitMQ dipilih karena konsep *exchange* dan queue-nya sederhana serta sesuai dengan pola *publish-subscribe* yang dibutuhkan. Selain itu, RabbitMQ menyediakan *management* UI bawaan untuk memantau kondisi queue, termasuk jumlah pesan yang tertunda ketika consumer tidak aktif.

2. Tipe exchange

Pilihan	*Topic exchange*
Alternatif	*Fanout exchange*
Alasan	*Fanout exchange* selalu mengirimkan seluruh event ke setiap consumer, sehingga consumer tidak dapat memilih jenis event yang diterima. *Topic exchange* tetap dapat mengirimkan seluruh event dan juga memungkinkan penyaringan dilakukan oleh broker. Dengan demikian, consumer baru yang hanya membutuhkan jenis event tertentu, misalnya event vulkanik, cukup berlangganan routing key yang sesuai.

3. Semantik pengiriman

Pilihan	*At-least-once*
Alternatif	*At-most-once*
Alasan	Pada *at-most-once*, event dapat hilang ketika consumer gagal sebelum selesai memproses pesan. Pada sistem

	ini, kehilangan peringatan bencana bagi tim lapangan lebih berbahaya daripada menerima peringatan ganda. Oleh karena itu, *at-least-once* dipilih dengan konsekuensi consumer harus bersifat idempoten.

2.5.3. Batasan Yang Disadari

Berikut beberapa hal yang secara sadar tidak ditangani pada rancangan Problem 5 beserta alasannya.

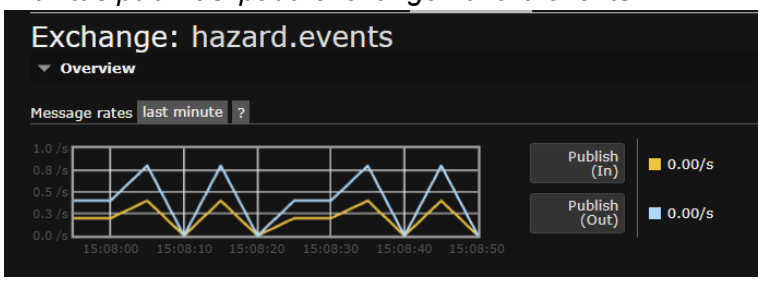
1. **Consumer baru tidak menerima event lampau**
   RabbitMQ hanya menyimpan pesan untuk queue yang sudah ada, sehingga consumer baru hanya menerima event yang dipublikasikan setelah queue-nya dibuat. Fitur replay tidak diterapkan karena tidak dibutuhkan spesifikasi.

2. **Data idempotensi tidak bertahan saat restart**
   Consumer Notifier menyimpan message_id yang sudah diproses di memori, sehingga data tersebut hilang ketika consumer dijalankan ulang. Akibatnya, pesan yang belum sempat dikonfirmasi sebelum consumer berhenti akan dikirim ulang oleh broker dan dapat menghasilkan notifikasi ganda. Penyimpanan persisten tidak diterapkan karena notifikasi pada sistem ini hanya dicatat ke log.

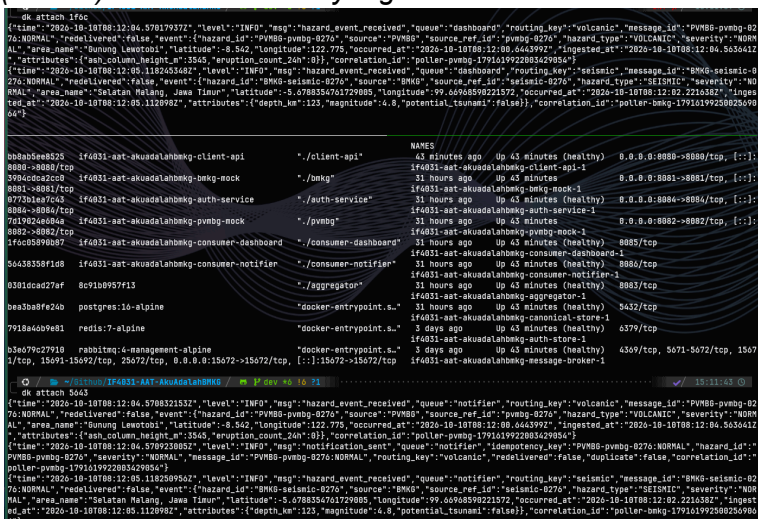
2.5.4. Bukti Penyelesaian

Berikut bukti terpenuhinya setiap kriteria penyelesaian Problem 5 berdasarkan pengujian yang dilakukan.

1. **Event dipublikasikan ke broker**

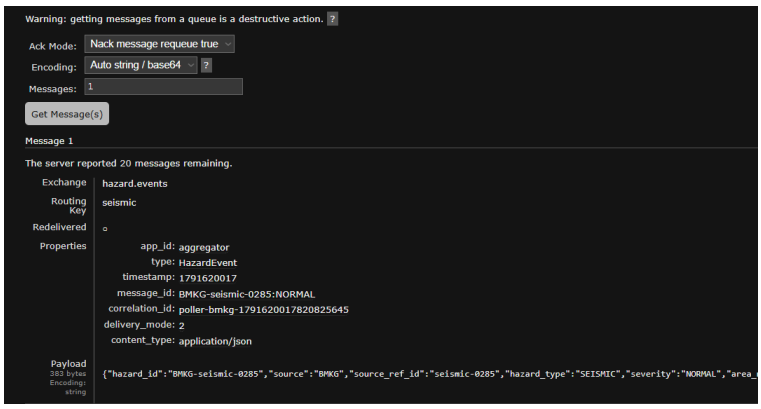
Pengujian	Sistem dijalankan, lalu aktivitas exchange `hazard.events` diamati melalui *management* UI RabbitMQ.
Hasil	Setiap `HazardEvent` baru tercatat masuk ke exchange `hazard.events`. *Aktivitas publikasi pada exchange hazard.events* 

2. Dua consumer menerima event yang sama

Pengujian	Log Consumer Dashboard dan Consumer Notifier dibandingkan untuk event yang sama.
Hasil	Kedua consumer menerima event dengan `message_id` dan `correlation_id` yang sama tanpa Aggregator mengetahui keberadaan keduanya. *Consumer dashboard (atas) dan consumer notifier (bawah) menerima event yang sama.* 

3. Consumer dihentikan sementara

Pengujian	Consumer Notifier dihentikan selama beberapa menit, lalu dijalankan kembali.
Hasil	Selama Consumer Notifier berhenti, Aggregator tetap mempublikasikan event tanpa tertahan dan Consumer Dashboard tetap menerima event. Pesan untuk Consumer Notifier menumpuk di queue-nya. Setelah dijalankan kembali, Consumer Notifier memproses seluruh pesan yang tertunda, sehingga tidak ada event yang hilang. *Consumer notifier dimatikan*  *Pesan tetap masuk dalam queue (queue notifier)*



Pemrosesan message pada queue setelah notifier hidup



4. Penambahan consumer baru

Pengujian	Consumer baru yang hanya berlangganan event vulkanik dijalankan tanpa mengubah maupun menjalankan ulang Aggregator.
Hasil	Queue baru terbentuk dan langsung menerima event vulkanik, sedangkan event seismik tidak diterima. Aggregator tetap berjalan tanpa perubahan kode. *Consumer baru menerima event vulkanik*



2.5.5. Broker dan Semantik Pengiriman

Sistem menggunakan RabbitMQ dengan pola *publish-subscribe* melalui *topic exchange*. Dalam mekanisme ini, Aggregator bertindak sebagai *publisher*, sedangkan setiap *consumer* memiliki queue masing-masing untuk menerima salinan event dari *exchange*.

Semantik pengiriman yang berlaku adalah *at-least-once*. Pesan dibuat *persistent* dan queue dibuat *durable* agar pesan tetap tersedia ketika consumer berhenti. Aggregator juga menunggu konfirmasi dari broker, sementara consumer baru memberikan konfirmasi setelah pesan selesai diproses. Dengan mekanisme tersebut, pesan tidak langsung hilang ketika consumer berhenti. Namun, pesan yang sama tetap dapat diterima kembali, misalnya ketika consumer berhenti setelah memproses pesan tetapi sebelum memberikan konfirmasi kepada broker.

Konsekuensi dari semantik pengiriman ini adalah consumer harus bersifat idempoten. Setiap pesan memiliki `message_id` yang dibentuk dari gabungan `hazard_id` dan `severity`. `severity` digunakan sebagai bagian dari kunci karena event yang sama dapat dipublikasikan kembali dengan tingkat ancaman yang lebih tinggi. Dalam kondisi tersebut, event tetap perlu diproses karena merupakan pembaruan dari event sebelumnya.

3.Lampiran

3.1. Penggunaan LLM

SURAT PERNYATAAN PENGGUNAAN LLM

Dengan ini, kami:

Nama/NIM : M. Rayhan Farrukh/13523035
Alfian Hanif Fitria Yustanto/13523073
Carlo Angkisan/13523091
Fakultas : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF4031
Nama Mata Kuliah : Arsitektur Aplikasi Terdistribusi
Judul Tugas : Tugas Besar Milestone 1 IF4031 Arsitektur Aplikasi Terdistribusi

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Adapun bentuk penggunaan dan alat yang digunakan adalah sebagai berikut:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan (Gemini)	Iya	Memeriksa tata bahasa, hasil pemeriksaan diperiksa kembali
Pembuatan Teks (Gemini, Claude)	Iya	Menggenerasi beberapa bagian teks untuk laporan yang kemudian dicek ulang dan disesuaikan lebih lanjut
Bantuan Diskusi dalam Penyusunan Konten (Gemini)	Iya	Bantuan memahami konsep materi, memvalidasi kebenaran pemahaman
Lainnya (Gemini, Claude)	Ya	Melakukan debugging, menemukan kode penyebab error. Setelah ditemukan, kode diperbaiki manual

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 10 Oktober 2026



M. Rayhan Farrukh
13523035

Bandung, 10 Oktober 2026



Alfian Hanif Fitria Yustanto
13523073

Bandung, 10 Oktober 2026



Carlo Angkisan
13523091